

Overview of VGG network

Subject: VGG network

1.

The VGGNets are a series of convolutional neural networks (CNNs) developed by the Visual Geometry Group (VGG) at the University of Oxford. The VGG family includes various configurations with different depths, denoted by the letter "VGG" followed by the number of weight layers. The most common ones are VGG-16 (13 convolutional layers + 3 fully connected layers, 138M parameters) and VGG-19 (16 + 3, 144M parameters). The VGG family were widely applied in various computer vision areas. An ensemble model of VGGNets achieved state-of-the-art results in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2014. It was used as a baseline comparison in the ResNet paper for image classification, as the network in the Fast Region-based CNN for object detection, and as a base network in neural style transfer. The series was historically important as an early influential model designed by composing generic modules, whereas AlexNet (2012) was designed "from scratch". It was also instrumental in changing the standard convolutional kernels in CNN from large (up to 11-by-11 in AlexNet) to just 3-by-3, a decision that was only revised in ConvNext (2022). VGGNets were rendered obsolete by Inception, ResNet, and DenseNet. RepVGG (2021) is an updated version of the architecture.

2.

Training The original VGG models were implemented in a version of C++ Caffe, modified for multi-GPU training and evaluation with data parallelism. On a system equipped with 4 NVIDIA Titan Black GPUs, training a single net took 2–3 weeks depending on the architecture.

3.

Convolutional modules: 3×3 convolutional layers with stride 1, followed by ReLU activations. Max-pooling layers: After some convolutional modules, max-pooling layers with a 2×2 filter and a stride of 2 to downsample the feature maps. It halves both width and height, but keeps the number of channels. Fully connected layers: Three fully connected layers at the end of the network, with sizes 4096-4096-1000. The last one has 1000 channels corresponding to the 1000 classes in ImageNet. Softmax layer: A softmax layer outputs the probability distribution over the classes. The VGG family includes various configurations with different depths, denoted by the letter "VGG" followed by the number of weight layers. The most common ones are VGG-16 (13 convolutional layers + 3 fully connected layers) and VGG-19 (16 + 3), denoted as configurations D and E in the original paper. As an example, the 16 convolutional layers of VGG-19 are structured as follows:

$$3 \rightarrow 64 \rightarrow 64 \rightarrow \text{downsample } 64 \rightarrow 128 \rightarrow 128 \rightarrow \text{downsample } 128 \rightarrow 256 \rightarrow 256 \rightarrow 256 \rightarrow \text{downsample } 256 \rightarrow 512 \rightarrow 512 \rightarrow 512 \rightarrow 512 \rightarrow \text{downsample } 512 \rightarrow 512 \rightarrow 512 \rightarrow 512 \rightarrow \text{downsample } \begin{array}{c} \left\{ \begin{array}{ccccccc} & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \\ & & & & & & \end{array} \right. \end{array}$$

where the arrow $c_1 \rightarrow c_2$ means a 3×3 convolution with c_1 input channels and c_2

output channels and stride 1, followed by ReLU activation. The $\rightarrow \text{downsample}$ means a down-sampling layer by 2x2 maxpooling with stride 2.

4.

Architecture The key architectural principle of VGG models is the consistent use of small 3×3 convolutional filters throughout the network. This contrasts with earlier CNN architectures that employed larger filters, such as 11×11 in AlexNet. For example, two 3×3 convolutions stacked together has the same receptive field pixels as a single 5×5 convolution, but the latter uses $(25 \cdot c^2)$ parameters, while the former uses $(18 \cdot c^2)$ parameters (where c is the number of channels). The original publication showed that deep and narrow CNN significantly outperform their shallow and wide counterparts. The VGG series of models are deep neural networks composed of generic modules: